# PE-JV-003

## Aula 04 - Microsserviços, Sagas & Consistência Distribuída

# Aquecimento

## Kahoot

Revisão da Aula 03: PostgreSQL, Índices B-Tree, Locks, UUIDv7 vs Sequence e ACID vs BASE.

Aguarde as orientações do professor.

## O Desafio de Engenharia

Na última aula, otimizamos o banco de dados para suportar altíssima concorrência. Porém, na arquitetura do marketplace, **Checkout, Pagamento e Estoque são serviços autônomos**, cada um com seu próprio banco de dados (sem `@Transactional` único).

**O incidente na Black Friday:**
Um cliente comprou uma Smart TV de R$ 3.000 via Pix. O serviço de Pagamentos debitou o valor imediatamente e confirmou a transação. No milissegundo seguinte, o serviço de Estoque esgotou as unidades e rejeitou a reserva.

**O problema:**
O cliente foi cobrado, o produto não existe para envio e o pedido travou em estado inconsistente. Como garantimos consistência entre serviços e bancos autônomos quando uma etapa falha no meio do caminho?

## A Missão do Dia

No seu documento de System Design (Aula 04):

- Avaliar o impacto da volumetria: quando uma transação local ou 2PC resolve e quando o padrão Saga é necessário.
- Definir e justificar o modelo de coordenação da transação distribuída (**Coreografia** ou **Orquestração**).
- Desenhar a máquina de estados e a **transação compensatória** (estorno automático), garantindo atomicidade no envio de eventos sem *Dual Write*.
- Desacoplar a leitura de status em alta escala da escrita transacional (**CQRS**) e definir a estratégia de testes de falha.

## Sala de Aula Invertida: A Pesquisa

Acesse os materiais compartilhados e pesquise:

1. **A Morte do 2PC:** Por que Two-Phase Commit e transações distribuídas tradicionais não escalam sob alto throughput? Em qual volume o 2PC ainda é válido?
2. **Coreografia vs Orquestração:** Em que momento a Coreografia vira "espaguete de eventos" e quando a Orquestração vira um monólito centralizador?
3. **Compensação Sem Rollback:** O que é uma transação compensatória e por que o consumidor precisa ser idempotente?
4. **Dual Write & Outbox:** O que acontece se o banco grava o pagamento mas a rede falha antes de publicar o evento no broker?
5. **CQRS:** Por que separar o modelo de escrita do modelo de consulta protege o sistema na Black Friday?

## O Desafio Arquitetural

No seu documento de System Design, fundamente suas decisões técnicas:

1. **Transações Distribuídas & Coordenação:**
   - Em quais cenários o 2PC basta e por que ele é inviável para o pico da Black Friday?
   - Qual modelo guiará sua Saga (**Coreografia** ou **Orquestração**) e como você mitigará os riscos específicos da sua escolha?
2. **Máquina de Estados, Compensação e Atomicidade:**
   - Quais são os estados do pedido no **Caminho Feliz** e no **Caminho de Compensação** (estorno automático)?
   - Como garantir que o cliente não fique no prejuízo se a própria tentativa de estorno falhar?
   - Como garantir atomicidade entre salvar o estado no banco e publicar o evento na fila sem sofrer com *Dual Write*?
3. **CQRS & Testes Transversais:**
   - Como separar a escrita do checkout da consulta massiva em "Meus Pedidos" lidando com a consistência eventual?
   - Como testar e comprovar na pipeline que a compensação automática funciona quando o estoque falha?

## Debate Técnico

É hora de debater com o comitê de engenharia (Staff Engineer):

- **Volumetria Real:** Se o sistema processasse 20 transações por dia, precisaríamos de Saga, Kafka e CQRS? Quando o 2PC basta?
- **Coreografia vs Orquestração:** Quais os trade-offs de governança, acoplamento e autonomia de cada escolha para o marketplace?
- **Estados Intermediários:** Como modelar o pedido enquanto a compensação está em andamento?
- **O Fim do Dual Write:** Como o Transactional Outbox elimina a perda silenciosa de eventos?
- **Consistência Eventual na Leitura:** Como o CQRS lida com a expectativa do usuário que atualiza a tela logo após pagar?

## Projeto Individual (System Design)

### Cenário de Engenharia (Compra Distribuída na Black Friday):

- **Serviços e Bancos Isolados:** Checkout, Pagamento e Estoque possuem bancos independentes operando a **5.000 TPS** no pico.
- **Incidente Real:** Pagamento via Pix aprovado, mas reserva de estoque rejeitada por SKU esgotado no milissegundo seguinte.
- **Pressão de Leitura:** Milhões de clientes atualizando a tela de "Meus Pedidos" a cada poucos segundos após o pagamento.

### Sua Missão:

- Responda às decisões do **Desafio Arquitetural** aplicadas diretamente ao cenário acima.
- Utilize o **Template da Aula 04** (disponível no LMS) e versione a entrega no repositório do seu projeto com commits atômicos e descritivos.

## Laboratórios Práticos (Opcional)

Disponibilizamos no LMS o guia de laboratórios e repositórios oficiais da indústria:

- **Temporal.io (Java SDK):** Execução do `Booking SAGA` e visualização da compensação automática na Web UI.
- **Debezium (Red Hat):** Ambiente Docker Compose de Transactional Outbox (Postgres + CDC + Kafka) sem Dual Write.
- **Confluent Developer:** Receita prática de Spring Boot com Dead Letter Queue (DLQ) e guias conceituais de CQRS.

## Próxima Aula: Estratégias de Entrega (Containers & CI/CD)

Agora temos microsserviços colaborando com consistência eventual e sagas. Como garantimos entregas seguras e escaláveis em produção?

**Tópicos Conceituais:**

- **Imutabilidade e Superfície de Ataque:** Imagens enxutas (distroless) e impacto no auto-scaling horizontal.
- **Controle de Blast Radius:** Decisões arquiteturais de rollout (Canary vs Blue-Green vs Rolling Update).
- **Supply Chain e Governança:** O papel do SBOM e portões de qualidade para bloquear vulnerabilidades (CVEs).

**Referências para Leitura Prévia:**

- 📄 Google Container Tools: Distroless
- 📄 Blue-Green Deployment (Martin Fowler)
- 📄 Canary Release (Martin Fowler)
- 📄 CISA: Software Bill of Materials (SBOM)

## Referências da Aula 04 (Microsserviços & Sagas)

Links oficiais para estudo e aprofundamento:

- 📄 **Padrão Saga:** Pattern: Saga (Chris Richardson)
- 📄 **Sagas em Sistemas Distribuídos:** Saga Pattern (Martin Fowler)
- 📄 **O Problema do Dual Write:** The Dual-Write Problem (Confluent)
- 📄 **Transactional Outbox:** Pattern: Transactional Outbox (Microservices.io)
- 📄 **Change Data Capture:** Debezium Architecture & CDC (Debezium.io)
- 📄 **CQRS:** Command Query Responsibility Segregation (Martin Fowler)
- 📄 **Consumidor Idempotente:** Idempotent Receiver (Enterprise Integration Patterns)

# Obrig.ADA